Phase 1 — Complete Learning Guide

Hindi (Model H) and Nepali (Model L)

Data Collection, Cleaning, and Tokenizer Construction, Explained From Scratch

Swaraj Kumar
2025202016

Personal study document — not a graded deliverable

August 21, 2026

What this document is

This is a self-study guide, written for myself, so that I can explain every part of my actual Phase 1 work confidently in a TA evaluation. It is **not** a course deliverable and does not affect grading. Every number in it comes from a real file this project actually produced — nothing is invented, and where something was not implemented, this document says so plainly instead of pretending otherwise. The graded report is `report/phase1/phase1_report.md` / `.tex` / `.pdf`; this document goes deeper, into theory and file-by-file mechanics, for my own understanding.

Contents

1 Introduction: What This Project Is, and Why **4**
- 1.1 The assignment, in plain words . . . 4
- 1.2 What Phase 1 actually delivers . . . 5
- 1.3 The two pipelines, and why they look completely different . . . 5

2 Core Theory: The Ideas Behind Every Stage **7**
- 2.1 What is a corpus? . . . 7
- 2.2 Manual vs. downloaded collection, and the 20% rule . . . 8
- 2.3 Unicode normalization: NFC vs. NFKC . . . 8
- 2.4 Language identification (Language ID / LID) . . . 9
- 2.5 Deduplication: exact vs. near-duplicate . . . 10
- 2.6 Tokenization: turning text into model input . . . 12
- 2.7 Train / validation / test splits, and data leakage . . . 14
- 2.8 Perplexity filtering: measuring how ”fluent” text reads . . . 15
- 2.9 Data quality control as a design philosophy . . . 15

3 The Complete Pipeline, End to End **16**
- 3.1 Big-picture flowchart . . . 16
- 3.2 Nepali’s stage order, drawn out . . . 16

4 Hindi Pipeline, File by File **18**
- 4.1 Overview . . . 18
- 4.2 download_hindi.py . . . 18
- 4.3 ingest_manual.py . . . 19
- 4.4 ingest_wikipedia_dump.py . . . 19
- 4.5 clean_hindi.py (using scripts/clean_lib.py) . . . 19
  - 4.5.1 The Hindi ↔ Bhojpuri cross-language story . . . 20
- 4.6 split_hindi.py . . . 21
- 4.7 train_tokenizer.py & analyze_tokenizer.py . . . 21

5 Nepali Pipeline, File by File **22**
- 5.1 The shared `corpuslib` package . . . 22
- 5.2 S00 – doctor.py: the lesson learned from a real incident . . . 22
- 5.3 S01 – collect.py / s01_collect.py . . . 23
- 5.4 S02 – calibrate.py . . . 23
- 5.5 S03 – normalize.py . . . 23
- 5.6 S04 – langid.py . . . 24

5.7 S07 – holdout.py (runs here, not after S06) . . . 24
5.8 S05 – heuristics.py . . . 24
5.9 S06 – perplexity.py . . . 24
5.10 S08 – dedup.py . . . 24
5.11 S09 – decontaminate.py . . . 25
5.12 S10 – balance_lanes.py . . . 25
5.13 S11 – finalize_splits.py . . . 25
5.14 S12 – train_tokenizer.py . . . 26
5.15 S13 – tokenizer_eval.py . . . 26
5.16 S14 – corpus_stats.py . . . 26
5.17 The outer loop: auto_grow_and_tokenize.py . . . 26

6 Data Flow: Raw Text to Final Corpus, With Real Numbers **28**
6.1 Corpus size and retention, both languages . . . 28
6.2 Manual vs. downloaded, and the final target . . . 28
6.3 Splits . . . 28
6.4 Tokenizer sweep . . . 28
6.5 The complete numeric journey . . . 28

7 Repository Structure **35**
7.1 Top-level layout . . . 35
7.2 Where each stage's output actually lives . . . 35

8 Deliverables Mapping **37**

9 Running the Pipeline From Scratch **39**
9.1 Hindi, step by step . . . 39
9.2 Nepali, step by step . . . 39
9.3 Troubleshooting . . . 40

10 TA Evaluation Question Bank **41**
10.1 Theory . . . 41
10.2 Pipeline . . . 42
10.3 Code and files . . . 43
10.4 Data and design decisions . . . 44
10.5 Hindi-specific . . . 45
10.6 Nepali-specific . . . 46
10.7 Limitations . . . 47

11 Cheat Sheet **48**
11.1 Numbers to have memorized . . . 48
11.2 File names to have memorized . . . 48
11.3 Pipeline stage order to recite . . . 48
11.4 Design decisions to be ready to defend . . . 49
11.5 One-sentence summary, if asked "what did you actually build in Phase 1?" . . . 49

12 Honest Limitations Summary **50**

Chapter 1

Introduction: What This Project Is, and Why

1.1 The assignment, in plain words

The course is called *Language Models and Agents*. The individual project asks every student to build **two completely independent Transformer language models from scratch** – meaning: write the model architecture yourself in PyTorch (no pretrained models, no HuggingFace model classes), collect your own training data, train your own tokenizer, and eventually train and evaluate two separate models.

Concept: Why two models, not one?

Because the whole point of the project is to see how a language model's behaviour changes with the amount of training data available. One model ("Model H") gets a **higher-resource** language – one where lots of clean text already exists on the internet. The other ("Model L") gets a **lower-resource** language – one where good text is scarce and has to be partly collected by hand. Comparing the two later (Phase 2/3) is the actual point of the exercise; Phase 1 is just building the raw material both models will eventually train on.

In this project: Language choice

Model H = Hindi. Hindi has an enormous amount of public text available (Wikipedia, curated corpora, web crawls), so reaching a large token count is realistic almost entirely from public sources.
Model L = Nepali. Nepali is on the assignment's fixed allowed list of lower-resource languages (Assamese, Bhojpuri, Bodo, Dogri, Konkani, Maithili, Manipuri, Mizo, Nepali, Sindhi). **Important personal note:** this project's *original* choice for Model L was **Bhojpuri**, and Bhojpuri was fully built, cleaned, and tokenized before Nepali replaced it once Nepali's own pipeline turned out to have enough public data to clear every target more comfortably. Bhojpuri's code has since been removed from the repository, but its numbers are still in the report as history – see Chapter 11 for the full story, so this is not glossed over.

1.2 What Phase 1 actually delivers

Phase 1 is **only** about data and the tokenizer – not the model itself. Concretely, for each language, Phase 1 must produce:

1. Scripts that collect raw text (both from the public internet and by manual effort).
2. A cleaning pipeline that turns messy raw text into a usable corpus.
3. A statistics report describing what was collected and how it was cleaned.
4. Train / validation / test splits of that cleaned corpus.
5. A trained subword tokenizer (and the code that trained it).

Note

Nothing about the neural network itself exists yet. There is no `model.py`, no training loop, no PyTorch `nn.Module`. That is Phase 2. If a TA asks "so where's the Transformer?" the honest answer for Phase 1 is: it doesn't exist yet, by design – this phase is the data foundation the model will be trained on later.

1.3 The two pipelines, and why they look completely different

This is one of the most important things to understand about this project, and also the thing most likely to confuse a first read of the code: **Hindi and Nepali are built by two genuinely different pieces of software**, not one script parameterised by language.

	Hindi	**Nepali**
Style	Single-pass cleaner: one script reads raw text and writes cleaned text in one straight run	Stage-based pipeline: 15 numbered stages (S00–S14), each its own script, each with its own checkpoint
Code location	`hindi/data/pipeline/` + `scripts/clean_lib.py`	`nepali/scripts/` + its own `corpuslib` package
Dedup method	Exact hash only	Exact hash *and* MinHash/LSH near-duplicate detection
Language filter	`langdetect` + a hand-written Bhojpuri-marker check	`fastText` with a margin-based decision
Tokenizer	SentencePiece Unigram	SentencePiece BPE

A detail that looks like a bug but isn't

Every Nepali stage script's docstring shows its example command as `python hindi/scripts/sXX.py` – but there is no `hindi/scripts/` directory anywhere in this repository. This is leftover template text: the `corpuslib` framework was designed so that *both* languages could share it, with Hindi meant to be the "source of truth" copy and Nepali a synced copy (see `run_pipeline.py`'s `sync` subcommand, which explicitly refuses to run from Nepali's side because "nepali is a copy, not the source"). In practice, that dual-language plan was never finished – Hindi kept its earlier, separate pipeline instead, and only Nepali actually uses this stage-based framework. If you go looking for `hindi/scripts/` because a docstring mentioned it, you won't find it, and that's expected.

Chapter 2

Core Theory: The Ideas Behind Every Stage

This chapter explains every concept used in this project **from zero** – assume you have never heard the term before. Each concept is followed by exactly how it shows up in this project's actual code.

2.1 What is a corpus?

Concept: Corpus

A **corpus** (plural: corpora) is just a large collection of text used to train or study a language model. It's not one document – it's potentially millions of documents (news articles, Wikipedia pages, books, social media posts) put together. The quality and size of the corpus is the single biggest factor in how good a trained language model ends up being – more so than almost any architecture choice made later in Phase 2.

In this project: This project's two corpora

Hindi's corpus, after cleaning, is 704,101 documents / 1,764,596,782 characters. Nepali's, after its full pipeline, is 1,918,265 documents / 635,204,333 tokens. Both are stored as one text file per document plus a metadata file (Hindi) or as line-delimited JSON files, one JSON object per document (Nepali) – two different storage conventions, again because the two pipelines were built independently.

2.2 Manual vs. downloaded collection, and the 20% rule

Concept: Why "manual" data matters

Anyone can write a script that downloads an existing dataset from Hugging Face. That is fast, but it does not demonstrate that a student can actually go out and *build* a dataset – find sources, extract text, clean up formatting, deal with things not already packaged for machine learning. The assignment requires that **at least 20% of the final training tokens, for both languages, come from manual collection** – meaning collection work beyond "point a downloader at an existing dataset."

Example: What counts as manual, what doesn't

Downloading `wikimedia/wikipedia` from Hugging Face: **not** manual – it's already a packaged dataset someone else prepared. Downloading the raw Wikipedia XML dump yourself from `dumps.wikimedia.org` and running it through WikiExtractor yourself: **is** manual, even though the underlying text is the same Wikipedia content – because you did the extraction work, not Hugging Face. This exact distinction is why `hindi/data/pipeline/ingest_wikipedia_dump.py` exists as a separate script from `download_hindi.py`: same content, different collection method, different `source_type` label.

In this project: Where manual/downloaded is tracked

Every single document, from the moment it's collected, carries a `source_type` field (`"manual"` or `"downloaded"`) in its metadata. This field survives all the way through cleaning to the final corpus, so the manual percentage can always be recomputed directly from real data rather than trusted as a remembered number. `scripts/validate_phase1.py` (Hindi) and `scripts/validate_phase1_nepali.py` (Nepali) both compute this percentage fresh, from the actual files, every time they run.
Hindi ended at 28.65% manual (by characters). Nepali ended at 26.41% manual (by tokens – Nepali's pipeline tracks tokens/words per lane, not characters, so its manual percentage is measured in a different unit; this is disclosed honestly in the report rather than papered over).

2.3 Unicode normalization: NFC vs. NFKC

Concept: Why text needs "normalizing" at all

The same visual character can sometimes be represented by more than one sequence of underlying bytes in Unicode. For example, a character with an accent mark can be stored as one single "precomposed" code point, or as two separate code points (the base letter, then a combining accent mark) that render identically on screen. If two documents contain the "same" text but encoded differently, a computer comparing them byte-for-byte would wrongly think they're different – and any deduplication step would fail to catch the duplicate. **Normalization** picks one canonical encoding so equivalent text always looks byte-identical.

Concept: NFC vs. NFKC – the difference that matters here

NFC (Normalization Form Canonical Composition) merges code-point sequences into their single composed form where possible, but preserves every character's actual identity and meaning. **NFKC** (Normalization Form Compatibility Composition) goes further – it also converts characters that are visually/semantically "compatible" into a simpler form (e.g. turning a stylized character variant into a plain one), which can *change* the text, not just re-encode it identically.

Example: Why NFKC is dangerous for Devanagari

Devanagari script (used by both Hindi and Nepali) uses two special invisible characters: **ZWNJ** (zero-width non-joiner, U+200C) and **ZWJ** (zero-width joiner, U+200D). These control whether two consecutive consonants visually merge into a single combined "conjunct" glyph or stay drawn as two separate letters – this is a real, meaningful spelling distinction in Devanagari, not decoration. NFKC's "compatibility" normalization can strip or alter these, silently changing how a word is actually spelled. NFC does not touch them.

In this project: This project's choice

Both pipelines use **NFC, never NFKC**. Hindi's `clean_lib.py:normalize_text()` calls `unicodedata.normalize("NFC", text)` and then strips only zero-width/control characters that are *not* ZWJ/ZWNJ (the regex `_ZERO_WIDTH_AND_CONTROL` explicitly excludes them). Nepali's `corpuslib/textnorm.py` makes the same choice, and even lets `configs/language.yaml` decide per-language whether to strip joiners at all (Hindi's config strips some; Nepali's does not, since ZWJ is load-bearing inside real Nepali conjuncts like पुर्‌याउने).

2.4 Language identification (Language ID / LID)

Concept: What language ID does

Language identification is the task of automatically detecting which human language a piece of text is written in. This sounds simple but is genuinely hard when two languages share the same script and a lot of vocabulary – exactly the situation between Hindi and Nepali (and historically, Hindi and Bhojpuri), since all three are written in Devanagari and share many common words.

Concept: Confidence vs. margin

A language-ID model doesn't just say "this is Hindi" – it gives a probability for every language it knows. A naive approach accepts a document if its top-predicted language's probability is above some threshold (e.g. 85%). The problem: for two very similar languages, a model can be, say, 60% confident it's Hindi and 55% confident it's Nepali – neither passes a raw-confidence threshold cleanly, but the **margin** between them (the gap between the top language's probability and the next-best competing language's probability) is a much more informative signal of genuine ambiguity than either raw number alone.

In this project: Two different approaches, by necessity

Hindi's pipeline uses Python's `langdetect` library, which **has no label for Bhojpuri at all** – it cannot classify text as Bhojpuri even if that's genuinely what it is, so it will happily call real Bhojpuri text "Hindi." To close that gap, Hindi's `clean_lib.py` adds a second, hand-written check: a list of common Bhojpuri words (बा, बाटे, गइल, etc.) versus common Hindi words (है, हैं, गया, etc.) – if Bhojpuri markers clearly outnumber Hindi markers, the document is rejected before `langdetect` even gets a say.
Nepali's pipeline uses `fastText`'s `lid.176` model instead, and explicitly computes the **margin** between Nepali's score and the best competing Devanagari-script language (Hindi, Maithili, Sanskrit, etc.) – see `corpuslib/langid.py`. This is a more principled version of the same underlying problem Hindi's marker-list check is trying to solve by hand.

Warning

An earlier version of this exact pipeline had a language-ID bug that deleted **94% of a corpus silently**: the `fastText` library crashed on a NumPy version incompatibility, the crash was caught by a blanket `except Exception`, and every document was treated as "rejected" – while the run still logged `SUCCESS`. This is exactly why `nepali/scripts/s00_doctor.py` exists: it tests every dependency actually works *before* a multi-hour run starts, and `s04_langid.py` now **aborts loudly** (rather than silently continuing) if the fraction of documents kept falls outside an expected band configured in `language.yaml`.

2.5 Deduplication: exact vs. near-duplicate

Concept: Why duplicate documents are a problem

If the same document (or a slightly reformatted copy of it) appears multiple times in a training corpus, the model effectively sees that text more often than it should, which can make it memorize that specific text rather than learning general language patterns. Large web-crawled datasets are full of duplicates – the same news article gets re-published on multiple sites, the same product description appears on a hundred pages, etc.

Concept: Exact-hash deduplication

The simplest approach: normalize each document's text (lowercase, collapse whitespace, strip punctuation), then compute a hash (a short fixed-length fingerprint) of the result. Two documents with the same hash are (almost certainly) exact duplicates once formatting differences are ignored. This is fast and simple, but misses documents that are *almost* the same but not byte-identical after normalization – e.g. the same article with one paragraph added.

Concept: Near-duplicate detection: MinHash + LSH

To catch documents that are similar but not identical, this project uses **MinHash** combined with **LSH (Locality-Sensitive Hashing)**.

MinHash, step by step: break a document into overlapping chunks of n consecutive words (called **shingles**, e.g. 5-word windows). Apply many different hash functions (this project uses 100) to the whole set of shingles, and for each hash function keep only the *minimum* hash value it produced. The resulting 100 numbers are that document's "signature." The mathematical guarantee is: the fraction of matching numbers between two documents' signatures is a good estimate of their real word-overlap similarity (the **Jaccard similarity** of their shingle sets) – without ever needing to compare the full texts against each other directly.

LSH then makes this scale to millions of documents: instead of comparing every document's signature against every other document's signature (which would be far too slow), the 100-number signature is split into bands (this project: 20 bands of 5 numbers each), and two documents are only ever compared for real if they land in the exact same bucket for at least one band. Similar documents are very likely to share at least one band; wildly different documents almost never do.

Example: Working through it with small numbers

Say a document has the shingles (3-word windows) {"ram went home", "went home today"}. Apply hash function #1 to both shingles, keep whichever result is smaller – that's the document's MinHash value #1. Repeat with 99 more hash functions. Two documents whose text overlaps heavily will keep landing on the same minimum value for many of those 100 hash functions purely because they share most of the same shingles to begin with – that's the whole trick.

In this project: Where this runs, and the manual-priority rule

Nepali's `corpuslib/dedup.py` implements exactly this: 100 permutations, 20 bands of 5 rows, a similarity threshold of 0.55, 5-word shingles. It runs twice: once in `s08_dedup.py` (removing duplicates *within* the corpus itself) and once in `s09_decontaminate.py` (checking that no training document is a near-duplicate of something already held out for validation/test – otherwise the model could effectively "see" its own test answers during training).

Hindi's pipeline only does **exact-hash** deduplication (`clean_lib.py:dedup_key()`) – it does not run MinHash/LSH at all. This is a real, acknowledged asymmetry between the two pipelines (Section 12), not a hidden gap.

One deliberate rule in both languages' dedup logic: when a duplicate is found between a manually-collected copy and a downloaded copy of the same text, **the manual copy always wins** and the downloaded copy is the one removed. Manual data is the scarce resource this project has to protect (it's what the 20% floor is measuring) – a "blind" dedup pass that didn't care which copy came from which lane could silently erode the manual percentage by deleting manual copies at random.

2.6 Tokenization: turning text into model input

Concept: Why models can't just read raw text

A neural network needs its input broken into a finite, fixed set of discrete units called **tokens**, each mapped to a number (an "embedding" index). You could use whole words as tokens, but that means the vocabulary has to include every possible word form ever seen – infeasible for morphologically rich languages like Hindi/Nepali, where a single root word can take dozens of suffixed forms. The modern answer is **subword tokenization**: break words into smaller, reusable pieces, so a large vocabulary of *whole* words is replaced by a much smaller vocabulary of *parts* that can be recombined.

Concept: Byte Pair Encoding (BPE)

BPE builds its vocabulary greedily: start with every individual character as a token. Repeatedly find the **most frequent adjacent pair** of existing tokens in the training corpus and merge that pair into one new token. Stop once the vocabulary reaches the target size. The key limitation: once two tokens are merged, that decision is **permanent** – BPE never revisits an earlier merge even if a later merge would have been a better overall segmentation.

Example: BPE merges, step by step

Suppose the corpus has many occurrences of the word "लड़का" (boy). BPE might first merge the two most frequent adjacent characters into one token, then merge that new token with the next most frequent neighbor, and so on – e.g. ल+ड़ → लड़, then लड़+का → लड़का becomes one single token once merges reach that far, *if* "लड़का" was frequent enough in the training data to earn that many merge steps before the vocabulary budget runs out.

Concept: Unigram Language Model tokenization

Unigram tokenization works differently: instead of building up from characters via merges, it starts from a large candidate set of possible subword pieces, then uses the EM (Expectation-Maximization) algorithm to iteratively estimate a probability for each candidate piece and discard the least useful ones, repeating until the vocabulary reaches the target size. Critically, for any given piece of text, Unigram considers the *whole space* of possible ways to split it into pieces and keeps whichever segmentation is most probable under the current model – it can effectively "reconsider" a segmentation choice that BPE's one-merge-at-a-time process never revisits.

In this project: Which algorithm each language uses, and why

Hindi uses SentencePiece Unigram. The argument (documented in `report/phase1/phase1_report.md` §5): Hindi is a morphologically rich, agglutinative-suffixing language on top of Devanagari's conjunct-cluster orthography (where one visual glyph can correspond to several underlying code points). BPE's inability to revisit a bad early merge is a real structural weakness for this kind of script; Unigram's probabilistic re-estimation is a better theoretical fit. This argument was **not empirically tested head-to-head on Hindi specifically** – it's a structural argument, not a measured one, and the report says so honestly.
Nepali uses SentencePiece BPE, chosen simply because that's what `nepali/configs/tokenizer.yaml` was configured to use as this pipeline's default – *not* a considered per-language linguistic decision. But there's a real safety net: the config includes `compare_unigram.enabled: true`, meaning *if* BPE's measured fertility (see below) had exceeded 2.10 tokens/word, the pipeline would automatically also train a Unigram model and keep whichever scored better. It never triggered – Nepali's BPE fertility stayed at 1.48–1.70 across the whole sweep, comfortably under that threshold – so this is a genuine comparison that was set up and simply didn't end up mattering, not an untested assumption.

Concept: Fertility, chars/token, and vocabulary utilization

Fertility = average number of tokens produced per whitespace- delimited word. Lower fertility means the tokenizer represents text more compactly (fewer tokens for the same content) – generally desirable, since it means more effective context length and less computation per unit of real text, but a fertility that's too low can mean the tokens are too coarse-grained to generalize well.
Characters per token is a related, more granular measure of compactness.
Vocabulary utilization = the fraction of the tokenizer's vocabulary that actually gets used at least once on held-out text. A vocabulary size larger than the language's real subword diversity wastes embedding-table parameters on pieces that are rarely or never emitted.

In this project: The actual trade-off that decided vocab size

Both languages were capped to a vocabulary size between 5,000 and 10,000 by a course-TA ruling, because a much larger vocabulary (e.g. 24,000) would make the embedding table alone consume roughly half of the whole ∼25M-parameter model budget planned for Phase 2 – leaving too little capacity for the actual Transformer layers. Within that 5k–10k range:

- **Hindi chose 8,000** on a fertility/embedding-cost trade-off: fertility kept improving from 5k to 10k, but the gain from 8k→10k (2.7% relative) was much smaller than 5k→8k (6.7% relative), for meaningfully more embedding cost. Measured directly: 8,000 produced *more* total tokens than 10,000 (505.9M vs. 491.9M), pushing Hindi to 101.2% of the 500M planning target instead of 98.4%.
- **Nepali chose 10,000** by its pipeline's own automated rule: "walk the swept sizes, stop at the first one where growing further buys less than a 3% relative fertility gain." No elbow was found within the swept range at all (fertility kept improving meaningfully all the way to 10,000), so the rule kept the largest candidate.

2.7 Train / validation / test splits, and data leakage

Concept: Why three separate splits

Train is what the model actually learns from. **Validation** is used during development to make decisions (like which tokenizer vocabulary size to pick) without ever touching the final judge of quality. **Test** is held back and used only once, at the very end, to report how well the finished model actually generalizes to text it has never influenced in any way – not during training, not during any tuning decision.

Concept: Data leakage

Leakage happens when information from validation or test data sneaks into training, or into a decision that shapes training – directly (the exact same document ends up in two splits) or indirectly (a near-duplicate of a test document ends up in train, so the model has effectively already "seen" it in a slightly different wrapper). Leakage makes evaluation numbers look better than the model's real, honest capability, which defeats the entire purpose of holding out test data in the first place.

Example: Splitting by document, not by sentence or line

If you split individual *sentences* from the same article across train and test, a model can learn stylistic and factual details from one sentence in train and then look unfairly capable when asked about a neighboring sentence from the same article in test – because it isn't really being tested on unseen content, just an unseen slice of a document it partly already saw. Both pipelines in this project split at the **whole-document** level specifically to avoid this.

In this project: How each pipeline actually splits

Hindi (`split_hindi.py`): shuffles all cleaned documents with a fixed random seed (42), then assigns roughly 96% to train, 2% to validation, 2% to test, purely by document count. Simple, but it runs *after* cleaning is fully done, so quality filtering has already happened to all three splits identically.

Nepali (`s07_holdout.py`) does something more careful, and it's one of the more interesting design choices in this project: val/test are carved out **immediately after language-ID (S04)**, *before* the heuristic quality filter (S05), the perplexity filter (S06), deduplication (S08), or lane balancing (S10) ever run. The reasoning: if the same quality filters that *select* training data also get to *curate* the evaluation set, then evaluation partly measures "does this model do well on text our own filters like," not genuine model quality on a fair, representative sample. `s05_heuristics.py` and `s06_perplexity.py` explicitly read S07's leftover `train_pool` output, never S04's output directly – so val/test genuinely never pass through those filters. The split itself is a deterministic hash of each document's ID against a fixed seed (not a shuffle held in memory), so the exact same split is reproducible from the seed alone, and it scales to a corpus that doesn't need to fit in memory to be split.

2.8 Perplexity filtering: measuring how ”fluent” text reads

Concept: What perplexity means, informally

Perplexity (or the closely related **cross-entropy**, its logarithm) is a measure of how ”surprised” a language model is by a piece of text – lower means the text follows patterns the model considers normal and predictable; higher means it's unusual, garbled, or statistically unlike the kind of text the model was trained to expect. A small n-gram model trained on clearly *well-written* reference text can be used as a crude but genuinely useful fluency detector: text that scores badly against it is more likely to be boilerplate, garbled extraction artifacts, or other low-quality noise, though it isn't a perfect signal.

In this project: Nepali's S06 stage

`s06_perplexity.py` trains a simple order-5 word n-gram model (with ”stupid backoff” smoothing) on a sample of documents from sources already known to be edited prose – Wikisource, OCR'd books. Every *other* document (not from those reference sources, to avoid scoring reference text against itself) gets a perplexity score, and the worst-scoring 3% is dropped as the least fluent tail. There's a built-in sanity check: if the dropped documents' *average length* turns out to be suspiciously short, that's the signature of a broken n-gram model flagging near-empty fragments rather than genuinely disfluent prose – in that case the whole filter skips itself rather than dropping real data for a reason that doesn't actually hold up. **Hindi's pipeline has no equivalent stage at all** – this is a capability only Nepali's pipeline has.

2.9 Data quality control as a design philosophy

Concept: ”Fail loudly” vs. ”fail silently”

A pipeline that silently drops data on an error (or, worse, silently accepts *everything* because a filter quietly stopped working) is far more dangerous than one that stops and complains – a silent failure can run for hours and produce a corpus that looks fine on the surface but is actually badly broken, and nobody notices until much later, if ever.

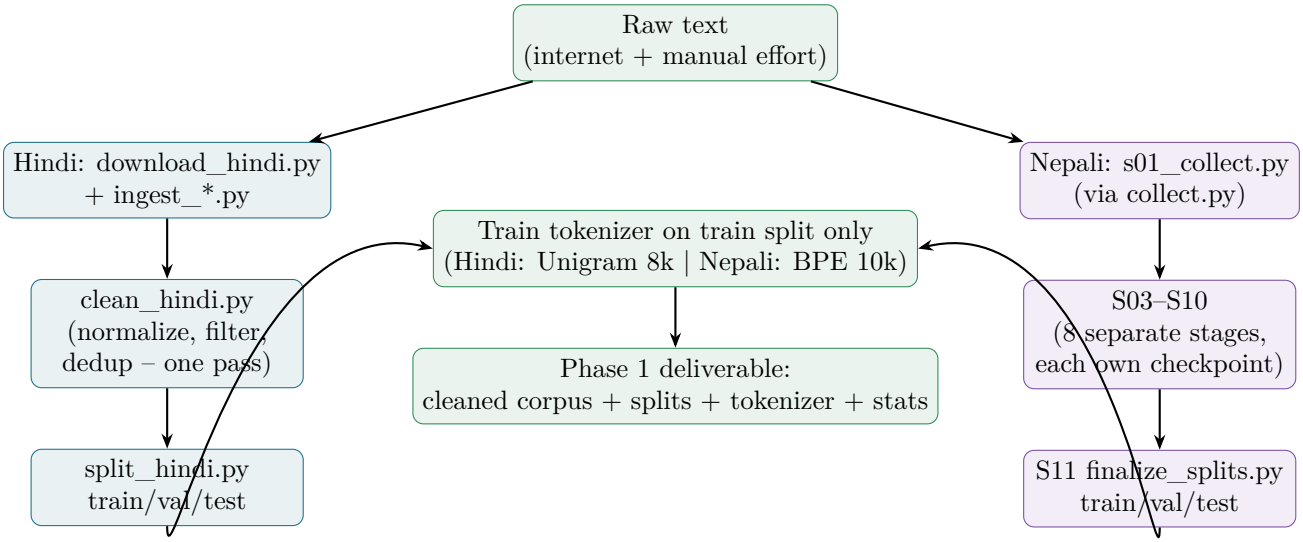
In this project: This is not theoretical for this project

The NumPy/fastText incident described in Section 2.4 (the 94%-of-corpus-silently-deleted bug) is exactly this failure mode, and it directly shaped how the Nepali pipeline is built now: `s00_doctor.py` tests every dependency before any expensive run starts; `s03_normalize.py` is designed so it can *never* drop a document (only later stages are allowed to decide ”good” vs. ”bad”, so retention numbers can always be attributed to one specific, identifiable cause); and `s04_langid.py`/`s05_heuristics.py` both check their own retention rate against an expected band from config and **abort the whole run** if it falls outside that band, rather than silently continuing on data that's probably wrong.

Chapter 3

The Complete Pipeline, End to End

3.1 Big-picture flowchart



3.2 Nepali's stage order, drawn out

The file numbers (S00…S14) do **not** match execution order – S07 genuinely runs before S05/S06, on purpose (Section 5.8). This diagram shows the *real* order.


S00 doctor – environment self-test (nothing processed yet)
S01 collect – pull every configured source
S02 calibrate – measure real tokens/word ratio
S03 normalize – NFC, never drops a document
S04 langid – fastText margin-based filter
S07 holdout – fork off val/test *right here*
S05 heuristics – length/symbol/repetition filters
S06 perplexity – drop worst 3% by fluency
S08 dedup – exact + MinHash/LSH near-dup
S09 decontaminate – train vs. held-out val+test
S10 balance_lanes – trim downloaded to target
S11 finalize_splits – write train/val/test.jsonl
S12 train_tokenizer – sweep BPE, pick by elbow
S13 tokenizer_eval – score on held-out test
S14 corpus_stats – one final numbers file

Chapter 4

Hindi Pipeline, File by File

4.1 Overview

Hindi's pipeline is five scripts plus one shared library, run one after another by hand (no orchestrator script ties them together the way Nepali's `run_pipeline.py` does):

Order	File	Purpose
1	`download_hindi.py`	Pull public sources (Wikipedia, Sangraha, FineWeb-2)
2	`ingest_manual.py`	Register typed/OCR'd/PDF/scraped manual text
2	`ingest_wikipedia_dump.py`	Register the self-extracted Wikipedia dump (manual)
3	`clean_hindi.py`	Normalize, filter, dedup – raw → cleaned
4	`split_hindi.py`	Cleaned → train/validation/test
5	`train_tokenizer.py`	Train SentencePiece Unigram
6	`analyze_tokenizer.py`	Measure fertility/UNK/coverage on validation

4.2 download_hindi.py

Input: nothing (streams directly from Hugging Face Hub). **Output:** `data/raw/downloaded/{text/*.txt, metadata.jsonl}`.

Streams four Hugging Face datasets rather than downloading them in full (each is far larger than needed): Wikipedia (`wikimedia/wikipedia:20231101.hi`), Sangraha's human-verified Hindi partition, Sangraha's larger but unverified partition, and FineWeb-2's CommonCrawl-derived Hindi partition. Each source function stops once it reaches a configurable character budget (defaults: no cap for Wikipedia since the dump is finite anyway; 300M chars for Sangraha verified; 500M each for Sangraha unverified and FineWeb-2). FineWeb-2 rows are additionally pre-filtered on the dataset's own `language_score` field (≥ 0.9) as a cheap first pass against CommonCrawl's known Hindi/Bhojpuri labeling noise – explicitly documented as *not* a substitute for the real language-ID filter that runs later in `clean_hindi.py`.

Every document gets a stable ID: `sha1("{source_name}: {raw_id}")`, truncated to 10 hex characters, prefixed `hi_downloaded_`. Stable IDs mean re-running with `--append` to add a new source never creates accidental duplicates of sources already on disk.

4.3 ingest_manual.py

Input: a folder of your own `.txt` files, or PDFs/images, or a file listing URLs. **Output:** same schema as above, written into `data/raw/manual/`.

One script, four subcommands, matching the four manual-collection methods this project's brief allows:

`typed` Registers plain text files you already typed or transcribed yourself.

`ocr` Runs Tesseract OCR over PDFs/images (needs the Tesseract binary installed separately, plus its Hindi `hin` trained data – `pip install pytesseract` alone is not enough).

`pdf` Extracts text directly from PDFs that already carry a text layer (no OCR needed) – skips a file if under 200 characters came out, since that's the signature of a scanned PDF with no real text layer, and tells you to use `ocr` for that case instead.

`scrape` Fetches a list of URLs you supply and strips boilerplate HTML (script/style/nav/header/footer tags) with BeautifulSoup.

Note

Every subcommand **skips** a source item that produces no usable text (a failed OCR, a 404'd URL) rather than padding it with anything – the brief explicitly forbids padding or duplicating text to inflate numbers, and this script never does either.

4.4 ingest_wikipedia_dump.py

Input: the output of running WikiExtractor yourself over a downloaded Wikipedia XML dump (a manual, one-time shell step, documented in the script's own docstring: `curl` the dump, then `python -m wikiextractor.WikiExtractor ... --json`). **Output:** `data/raw/manual/` – this is the **single largest manual source for Hindi**, 160,591 documents / 234.1M characters.

This script only ingests the already-extracted JSON output into this project's schema; it doesn't run WikiExtractor itself. It writes using a **file-lock-protected** append (`append_metadata_locked`, using an atomic `os.O_CREAT|os.O_EXCL` lock file) rather than a plain append, because this script's output directory can be written concurrently by other manual-ingestion processes – an earlier version of this project had two writers interleave lines mid-write into the same metadata file and corrupt it.

4.5 clean_hindi.py (using scripts/clean_lib.py)

Input: `data/raw/{manual,downloaded}/`. **Output:** `data/interim/`, `data/cleaned/`, `data/metadata/clean_stats.json`.

This is the heart of Hindi's pipeline, and it's a genuinely important detail that **manual documents are processed before downloaded ones** (`RAW_DIRS = [raw/manual, raw/downloaded]`) – deduplication keeps whichever copy it sees *first*, so this ordering is what guarantees a manual copy always wins a cross-source duplicate tie, never the downloaded copy. The pipeline, per document:

1. **Normalize** (NFC + strip non-Devanagari-meaningful invisible characters) via `clean_lib.normalize_text()`.
2. **Strip pure-Latin tokens** (`strip_latin_tokens()`) – removes whitespace-delimited tokens that are *purely* Latin-script (e.g. stray English words, URLs), but leaves digits and mixed tokens (like "5G", "iPhone14") untouched, since only foreign-script noise is meant to be removed, not numbers.
3. **Drop malformed documents** – if more than 1% of characters are the Unicode replacement character (a sign of a decoding failure / mojibake), the whole document is dropped.
4. **Drop too-short documents** – under 200 characters after cleaning.

→ **data/interim/** snapshot written here – text that passed the cheap structural checks, before language-ID or dedup.

5. **Language filter** (`language_ok()`) – three checks in order: (a) Devanagari-character ratio ≥ 0.5; (b) the Bhojpuri lexical-marker check described in Section 5.6 – an **active reject filter**, not just a measurement; (c) `langdetect`'s own verdict at $\geq 85\%$ confidence for Hindi.
6. **Deduplicate** – exact hash of normalized, punctuation- stripped, lowercased text (`dedup_key()`); first-seen document for a given hash wins.

Real drop counts: of 773,606 raw documents, 17 dropped as malformed, 63,132 as too short, 228 for low Devanagari ratio, 1,365 as "likely Bhojpuri," 1,778 on a `langdetect` mismatch, 2,983 as exact/near duplicates – leaving 704,103 cleaned (later corrected to 704,101 after a separate cross-language fix, see Section 4.5.1).

In this project: Two engineering details worth knowing

Single-instance locking: `acquire_single_instance_lock()` refuses to let a second copy of `clean_hindi.py` run concurrently against the same output directories, using an OS-atomic `O_CREAT|O_EXCL` lock file rather than a "check if a lock file exists, then write one" pattern – the latter has a race condition where two processes launched close together can both pass the "does it exist" check before either has written the file. This exists because this project's development environment was observed to sometimes launch two OS processes for what looks like one background command.
Resume support: `--resume` lets an interrupted multi-hour run continue from where it stopped rather than restarting from document zero, by reconstructing which documents were already processed (from `data/interim/`) and which deduplication hashes were already seen (from `data/cleaned/`'s stored `dedup_key` field) – without needing to reopen hundreds of thousands of individual small text files one at a time, which was measured to make an early version of resume take 100 minutes wall-clock for only 6 minutes of real work.

4.5.1 The Hindi ↔ Bhojpuri cross-language story

This is real project history worth being able to explain. While Bhojpuri was still this project's Model L, `scripts/validate_phase1.py`'s cross-language check found **2 documents** whose content hash was identical between the Hindi and Bhojpuri cleaned corpora – genuinely Bhojpuri news text (containing Bhojpuri-specific verb forms like कहलन, जइहें) that `ai4bharat/sangraha` had mislabeled as

Hindi (`hin`) upstream, while `HuggingFaceFW/fineweb-2` correctly labeled the same underlying text as Bhojpuri (`bho_Deva`). Both documents were removed from Hindi's cleaned corpus with a small, targeted, one-time script (`scripts/remove_cross_language_duplicates.py`) rather than a full pipeline rerun – which is why Hindi's final count is **704,101**, two less than `clean_stats.json`'s own recorded 704,103. This correction is permanent and has nothing to do with Bhojpuri's later removal from the project – it's just a real, documented data-quality fix in Hindi's own history.

4.6 split_hindi.py

Input: `data/cleaned/`. **Output:** `data/splits/{train,validation,test}/text/` + 3 manifest JSON files.

Shuffles cleaned document metadata with a fixed seed (42), assigns ∼96%/2%/2% by document count, and **hardlinks** (not copies) each document's text file into its split folder – an important performance detail: on this project's Windows/NTFS filesystem, copying ∼500k+ individual small files one syscall at a time was measured at about 2,500 files/minute, which would take hours; hardlinking (an instant, byte-identical filesystem operation, when source and destination are on the same volume) avoids that cost entirely, falling back to a real copy only if hardlinking fails.

4.7 train_tokenizer.py & analyze_tokenizer.py

Input: `data/splits/train/` (train_tokenizer.py) / `data/splits/validation/` (analyze_tokenizer.py) + a trained `.model` file. **Output:** `hindi_unigram_{5000,8000,10000}.model/.vocab`, and printed evaluation metrics.

Trains SentencePiece Unigram with fixed settings across the whole sweep (`character_coverage=0.9995`, `byte_fallback=True`, `input_sentence_size=2,000,000` with shuffled sampling – Hindi's train split is large enough that this samples only about 22% of it per training run, a standard SentencePiece cost-bounding mechanism, not a shortcut specific to this project). `analyze_tokenizer.py` then measures fertility, chars/token, UNK rate, byte-fallback rate, and vocabulary utilization purely on the **validation** split – never train, never test – so the numbers used to pick a vocab size are a genuine held-out measurement.

Chapter 5

Nepali Pipeline, File by File

5.1 The shared `corpuslib` package

Every Nepali stage script imports from `nepali/scripts/corpuslib/` – a small internal library, not a public package. Knowing what each module is for makes the individual stage scripts much easier to read:

Module	What it provides
`config.py`	Loads `language.yaml/sources.yaml`; every path (raw/interim/final/tokenizer dirs) is a prop
`schema.py`	Shared field names/constants (e.g. `LANE_MANUAL`, `content_hash`, `count_words`)
`stageio.py`	Reads/writes one stage's JSONL output + its manifest JSON; every stage uses this, never raw
`textnorm.py`	NFC normalization, PII redaction (email/phone)
`langid.py`	Wraps fastText, implements the margin-based accept/reject decision
`heuristics.py`	The length/symbol/repetition quality checks S05 applies
`ngramlm.py`	The small n-gram model S06's perplexity filter trains and scores against
`dedup.py`	MinHash signature computation + LSH bucket matching, used by both S08 and S09
`ledger.py`	Tracks manual vs. downloaded word/token totals; `cap_downloaded_lane()` is what S10 calls
`nethttp.py`	A rate-limited HTTP session (`PoliteSession`) used by S01's collectors
`rawtrim.py`	Trims the raw **downloaded** lane down to a target size before a rebuild (never touches manua
`runlog.py`	Writes the structured, human-readable entries you see in `log.md` – one line per stage run
`jsonlio.py`	Low-level atomic JSON/JSONL read-write helpers

Note

`test_corpuslib.py` (357 lines) is a real unit-test file covering this shared library – worth knowing it exists if a TA asks whether any part of this pipeline has automated tests.

5.2 S00 – doctor.py: the lesson learned from a real incident

Input: nothing (pure environment check). **Output:** a PASS/WARN/FAIL report, no data touched.

This script exists because of one specific, documented failure: the `fasttext` PyPI wheel calls a NumPy function (`np.array(probs, copy=False)`) that raises an error on NumPy ≥ 2. An earlier

version of this pipeline caught that exception with a blanket `except Exception: dropped += 1` and proceeded to delete **2,275,943 Hindi and 1,710,844 Nepali documents while logging SUCCESS** – a one-line environment incompatibility that took four hours of pipeline time to surface, and was initially misread as a data problem rather than an environment problem. Every check in `s00_doctor.py` maps to a specific way the pipeline could silently produce a wrong corpus; a failing check prints the exact command that fixes it.

5.3 S01 – collect.py / s01_collect.py

Input: `configs/sources.yaml` (which sources, what target word count each). **Output:** `data/raw/{manual,downloaded}/`.

Sources run **concurrently in a thread pool** – since this is network-bound work (downloading/scraping), not CPU-bound, threads are genuinely effective here. `corpuslib.nethttp.PoliteSession` rate-limits *per host*, so a slow domain only throttles itself, not every other source running in parallel. No single source failure can abort the whole run – every source is wrapped so any exception becomes a recorded `FAILED` result for that source alone, which is the whole point of letting this run unattended overnight. Progress is written to `data/collection_status.json` every few seconds (with a 30-second heartbeat even when nothing has changed) so the run can be inspected live from another terminal.

In this project: Real sources for Nepali

Manual lane: `news_onlinekhabar` (the largest single source, 91.6% of manual tokens), `wikipedia_api`, `news_bbc_nepali`, `wikibooks_api`, `wiktionary_api`, and several smaller news sites. Downloaded lane: `ai4bharat/sangraha` (verified + unverified) and `ai4bharat/IndicCorpV2`.

5.4 S02 – calibrate.py

Input: a sample of raw collected text (up to 200M characters). **Output:** `configs/token_calibration.json`.

Every word-count target in `language.yaml` is written as a number of *words*, but the pipeline actually needs to reason in *tokens* once a real tokenizer exists. Guessing the words-to-tokens ratio ahead of time is exactly what produced unusable budgets in an earlier version of this project's planning. This step trains a **throwaway** SentencePiece BPE model (8,000 vocab, not one of the real candidate tokenizers – S12 trains those, on cleaned text, sweeping real vocab sizes) purely to measure this ratio directly. **Measured: 1.5451 tokens/word.** There's a plausibility check too (the factor must land between 1.3 and 2.6) – a value outside that range would mean something is wrong with the sample, not a number to trust blindly.

5.5 S03 – normalize.py

Already covered in depth in Section 3.3 (Unicode theory). One detail worth repeating here: this is also where each document's permanent `doc_id` is computed, as a content hash of the **normalized** text – not the raw collection-time ID, since no real normalization has happened yet at collection time. The original collection-time ID is kept at `meta.raw_doc_id` so it's never lost, just superseded as the primary key.

5.6 S04 – langid.py

Already covered in depth in Section 3.4. One number worth remembering: 99.78% retention (2,655,504 → 2,649,643 documents) – very high, because most of what was collected was already reasonably Nepali to begin with; the filter exists to catch the exceptions, not to do heavy lifting on a badly-mixed corpus.

5.7 S07 – holdout.py (runs here, not after S06)

Already covered in depth in Section 3.7. The mechanism, briefly again: a deterministic hash of each document's ID against a fixed seed (`blake2b(f"{seed}:{doc_id}")`) buckets it into train/val/test without needing the whole corpus in memory. **Result: train-pool 2,251,378 documents**, feeding S05/S06; val and test are set aside completely untouched by any quality filter from this point forward.

5.8 S05 – heuristics.py

Input: S07's `train_pool` output. **Output:** `data/interim/s05_heuristics/`.

Checks length, word count, symbol ratio, Devanagari ratio, repeated-line ratio, mean word length, sentence-terminator presence, URL density, and lexical diversity – every threshold read from `configs/cleaning.yaml`, so Hindi and Nepali can (and do) use different thresholds without touching this shared code. Nepali's thresholds are deliberately **conservative** (every dropped document is unrecoverable, unlike Hindi's surplus of data). **Result: 99.73% retention.** 20 kept and 20 dropped example documents per drop reason are saved to `data/interim/filter_samples/` so every threshold choice is reviewable against real text, not just a retention percentage.

5.9 S06 – perplexity.py

Already covered in depth in Section 3.9. **Result: trained on 5,000 reference documents (Wikisource, OCR'd books); dropped the worst 3% of 1,383,607 scored documents, cutoff at 19.42 bits/token.** Runs in two passes over the data (score everything to find the cutoff, then re-score and apply it) so memory stays flat regardless of corpus size – only per-document numeric scores are held in memory across the whole corpus at once, never full document text.

5.10 S08 – dedup.py

Already covered in depth in Section 3.6. **Result: removed 52,084 of 2,178,418 documents** (2,350 exact + 49,734 near-duplicates); **21,273 cross-lane duplicate pairs found, all 21,273 resolved to the manual copy** per the priority rule. MinHash signature computation (the expensive part – 100 hash permutations per document) is parallelized across CPU cores; the actual LSH bucket insertion stays single-threaded, since that ordering is what makes the manual-priority rule deterministic (this is why manual documents are always processed first internally too, same reasoning as Hindi's `RAW_DIRS` ordering).

5.11 S09 – decontaminate.py

Already covered in depth in Section 3.7 (leakage). Uses the exact same MinHash machinery as S08, but in the other direction: builds one LSH index over the held-out val+test documents, then checks every train document against *that* index. Anything above the similarity threshold is removed from **train only** – val/test are never touched here, they're the fixed reference something in train might be leaking from. **Result: removed 17,111 train documents; a final rescan afterward found 0 remaining above-threshold matches** – the script explicitly treats that rescan as a real pass/fail check, not just a reported number.

5.12 S10 – balance_lanes.py

Input: S09's decontaminated output. **Output:** `data/interim/s10_balanced/`, `data/interim/balance_report.json`.

Concept: Absolute targets instead of a ratio – why this matters

An earlier version of this exact pipeline tried to hit a manual *ratio* (e.g. "manual must be 24% of the total") by shrinking the downloaded lane until that percentage was reached. The problem: when the manual lane was genuinely small (5.82% before any correction), chasing a percentage target meant throwing away a huge amount of perfectly good downloaded text (209.8M words cut down to 41.1M) *just to satisfy a ratio* – conflating "how much text do we have" with "what fraction is manual," which are not the same problem. `s10_balance_lanes.py` instead works off **two independent absolute token targets**: manual $\geq$ 100M tokens, downloaded $\geq$ 400M tokens.

The manual lane is kept at **100%, unconditionally** – it is never subsampled here, since it's the scarce resource. The downloaded lane is trimmed down to approximately its 400M-token target, converted to a word budget using S02's measured tokens-per-word factor. If manual still falls short of its own target after this, that's logged as a **WARNING pointing at collection** (go collect more manual text), never a reason to cut *less* downloaded text than the target calls for. **Result: manual share raised from 23.27% to 27.42%** of the train pool by trimming 589,223 downloaded documents.

5.13 S11 – finalize_splits.py

Input: S10's balanced train + S07's untouched val/test. **Output:** `data/final/{train,val,test}.jsonl`, `data/final/split_stats.json`.

Writes the actual final files this project's tokenizer and (eventually) model will read. Reports manual ratio **per split**, not just overall – flagging any split whose manual share deviates by more than 10 percentage points from the overall figure, since lane balancing only targets the train pool specifically; val/test could in principle drift if collection itself happened to be lane-skewed toward one split, and that would silently bias evaluation without this check. **Result: 1,520,000 / 199,428 / 198,837 documents, zero document-id overlap across all three splits.**

5.14 S12 – train_tokenizer.py

Already covered in Section 3.10 (Unigram-comparison safety net) and Section 3.11 (vocab-size selection). **Input:** `data/final/ train.jsonl` only. **Output:** `nepali_bpe.model/.vocab` + the full sweep under `tokenizer/sweep/`. `character_coverage=0.9995` is set explicitly because SentencePiece's default is tuned for Latin scripts and under-covers Devanagari's combining-mark range otherwise. `byte_fallback` is **disabled on purpose** here – with it enabled, every byte sequence becomes representable and UNK rate is identically zero at every vocabulary size regardless of real coverage, which would make the assignment's required "unknown-token statistics" meaningless.

5.15 S13 – tokenizer_eval.py

Input: the chosen tokenizer + `data/final/test.jsonl` (genuinely held-out, never touched by any selection decision). **Output:** `evaluation_report.json`. **Result: test fertility 1.4672, test chars/token 4.405, test UNK rate 0.0%.**

5.16 S14 – corpus_stats.py

Input: every stage's own manifest file + the final splits with real token counts from S13. **Output:** `data/final/corpus_stats.json` – the single file that S15's report generator and S16's QA gate both read, so nothing gets measured twice under two different definitions that could quietly disagree. **Result: 635,204,333 total tokens, both absolute lane targets independently confirmed met.**

5.17 The outer loop: auto_grow_and_tokenize.py

Input: the current state of raw data on disk. **Output:** a fully-rebuilt, target-meeting corpus, plus `growth_log.json` / `growth_summary.json`.

This script is not one of the numbered stages – it's an outer driver written specifically to close the gap between "run the pipeline once" and "actually hit both absolute token targets," without ever loosening a cleaning filter to manufacture volume. Its logic:

1. **One-time only**, before round 1: trim raw **downloaded** data down to roughly the target size (never touches raw manual), and wipe `data/interim/`, `data/final/`, `tokenizer/` so the rebuild starts clean.
2. Run the full S02–S14 pipeline.
3. Read `corpus_stats.json`'s real, measured totals. Check both absolute targets independently.
4. For whichever lane (manual or downloaded, or both) is still short, raise that lane's `target_words` in `sources.yaml` (up to a hard per-source ceiling, so this can't runaway-fill the disk) and re-run `collect.py` for just that lane – its own resume/ already-collected tracking guarantees this never re-downloads anything already on disk.
5. Repeat from step 2, up to 4 total rounds. A lane growing by less than 5% between two consecutive rounds is treated as having hit its real ceiling and is not grown further.

This is the mechanism that actually got Nepali from an initial undershoot to **127.0% of the 500M planning target**, and it's a genuinely different strategy from Hindi's pipeline, which has no equivalent auto-growth loop at all – Hindi's source character budgets were set once, by hand, in `download_hindi.py`'s CLI defaults.

Chapter 6

Data Flow: Raw Text to Final Corpus, With Real Numbers

Every figure in this chapter is one of the actual figures generated by `report/phase1/make_plots.py` for the graded report – nothing here is redrawn or re-imagined, these are the literal same PNG files.

6.1 Corpus size and retention, both languages

6.2 Manual vs. downloaded, and the final target

6.3 Splits

6.4 Tokenizer sweep

6.5 The complete numeric journey

Stage	Hindi documents	Nepali documents
Raw collected	773,606	2,655,504
After language-ID	(part of cleaning below)	2,649,643
After quality filters	(part of cleaning below)	2,178,418
After deduplication	(part of cleaning below)	2,126,334
After decontamination	–	2,109,223
Cleaned / final train pool	**704,101**	**1,520,000 (train only)**
Final train	675,937	1,520,000
Final validation	14,082	199,428
Final test	14,082	198,837
Final total documents	**704,101**	**1,918,265**
Final total tokens	**505,852,028**	**635,204,333**
% of 500M target	**101.2%**	**127.0%**

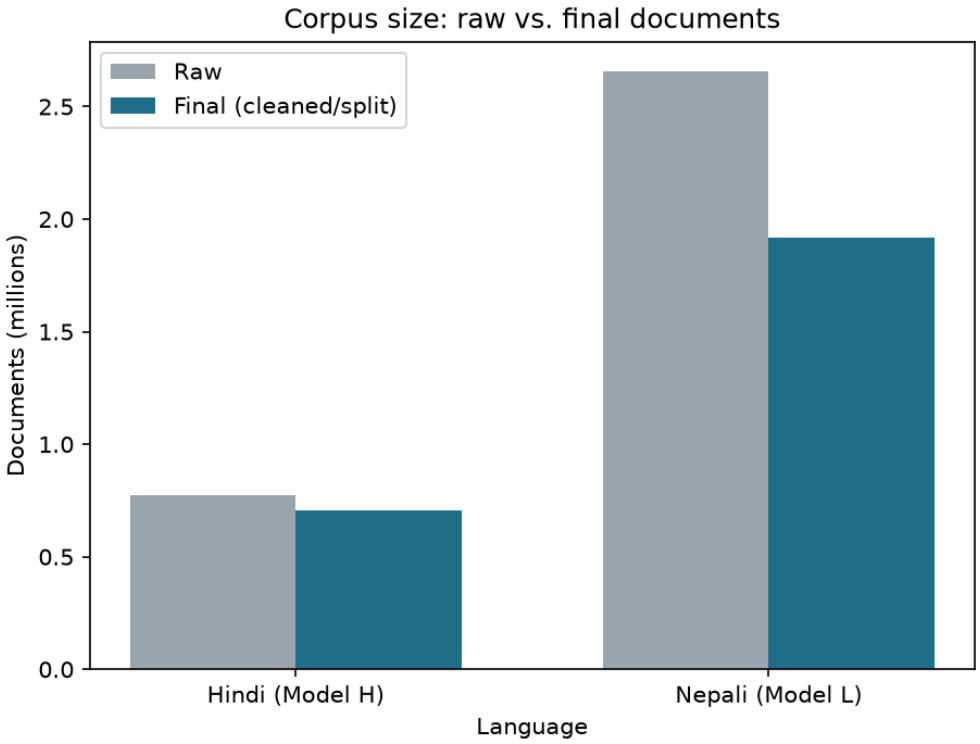


Figure 6.1: Raw vs. final document counts, both languages. Hindi's retention (91.0% of raw documents kept) is much higher than Nepali's overall pipeline, because Nepali's stage-based pipeline applies more filters (quality, dedup, decontamination, lane balancing) than Hindi's single-pass cleaner does.

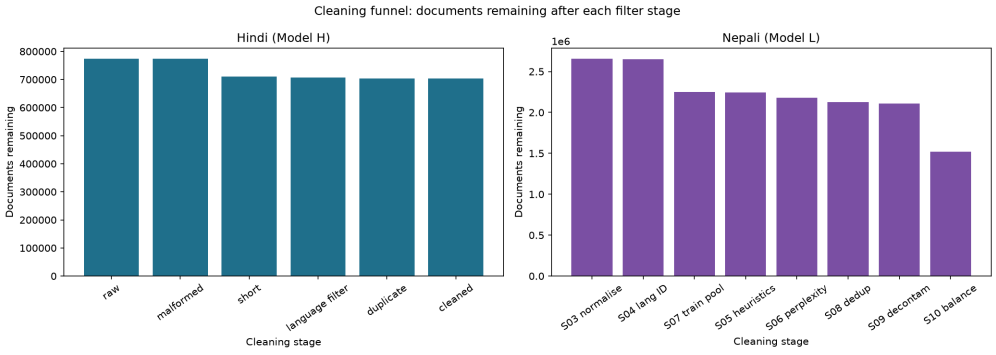


Figure 6.2: Document retention through each pipeline's own stages. Note the two subplots use completely different stage vocabularies – Hindi's five stages vs. Nepali's eight – because the two pipelines genuinely measure different things, not because one is missing information.

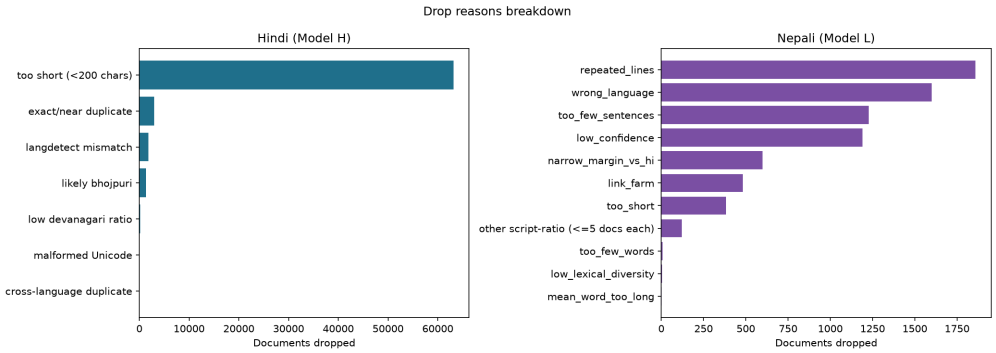


Figure 6.3: Every dropped document, broken down by the specific reason it was dropped, for both languages.

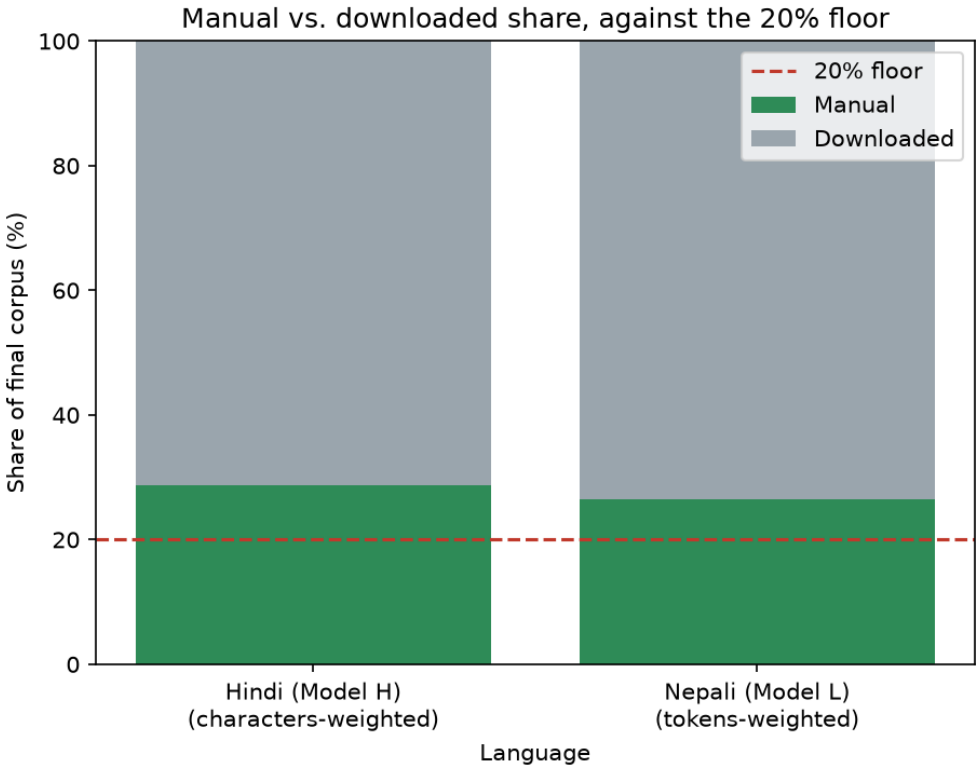


Figure 6.4: Manual vs. downloaded token share for both languages, against the 20% floor both must clear. Note the axis label: Hindi is char-weighted, Nepali is token-weighted – a real methodology difference, not an error.

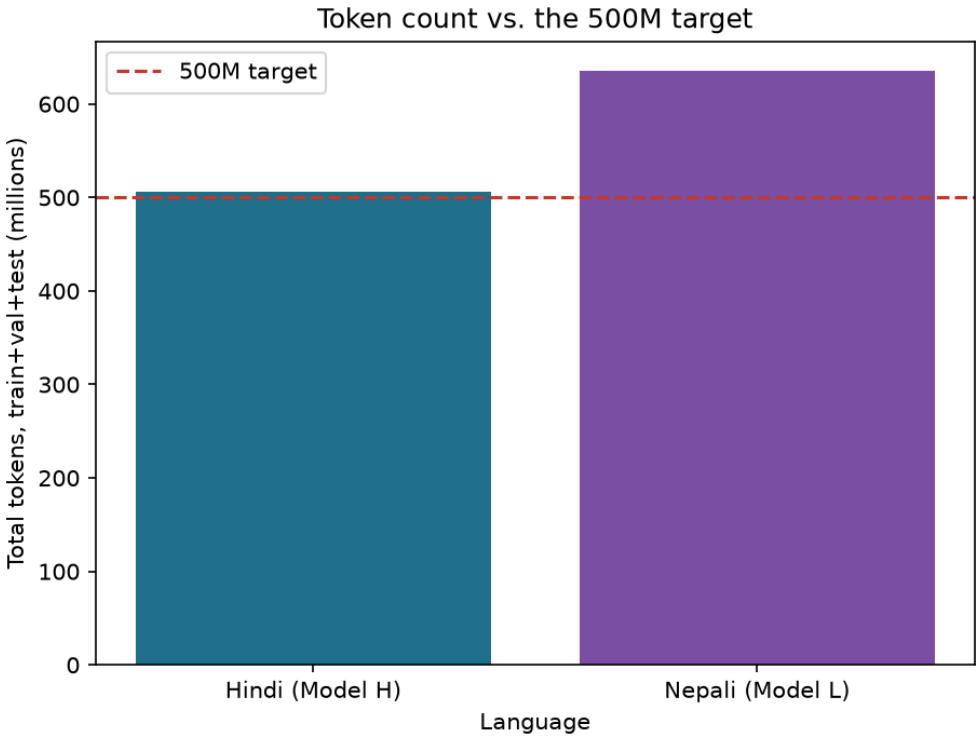


Figure 6.5: Final token count vs. the assignment's ∼500M planning target. Hindi reaches 101.2%; Nepali reaches 127.0%.

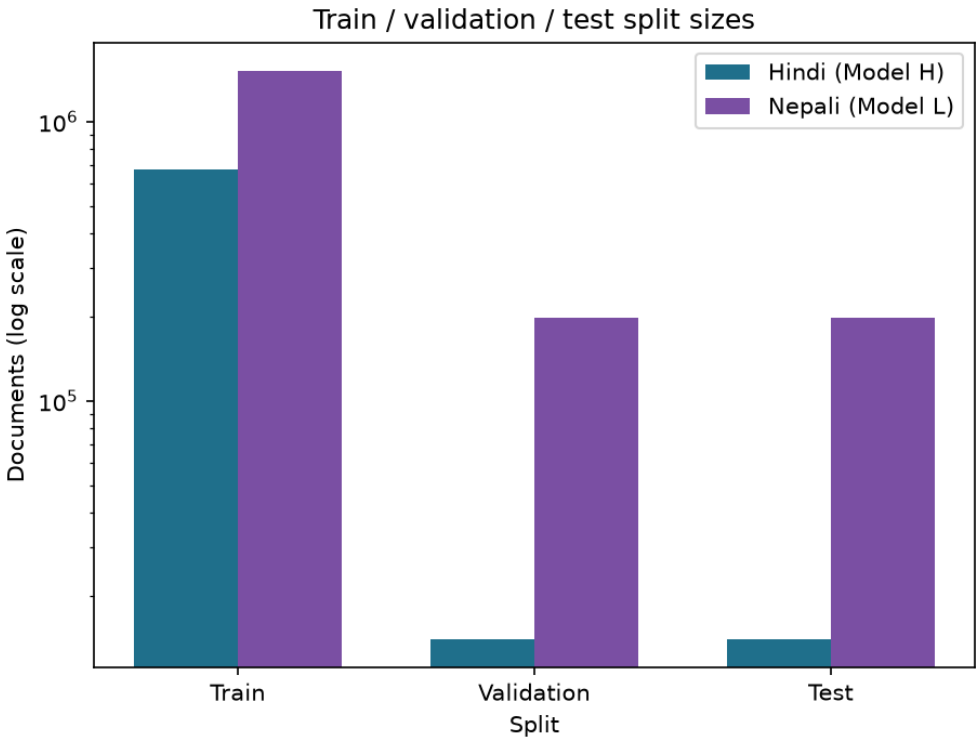


Figure 6.6: Train/validation/test document counts, both languages, log scale (train dwarfs val/test by roughly 8x for Hindi and 7.6x for Nepali).

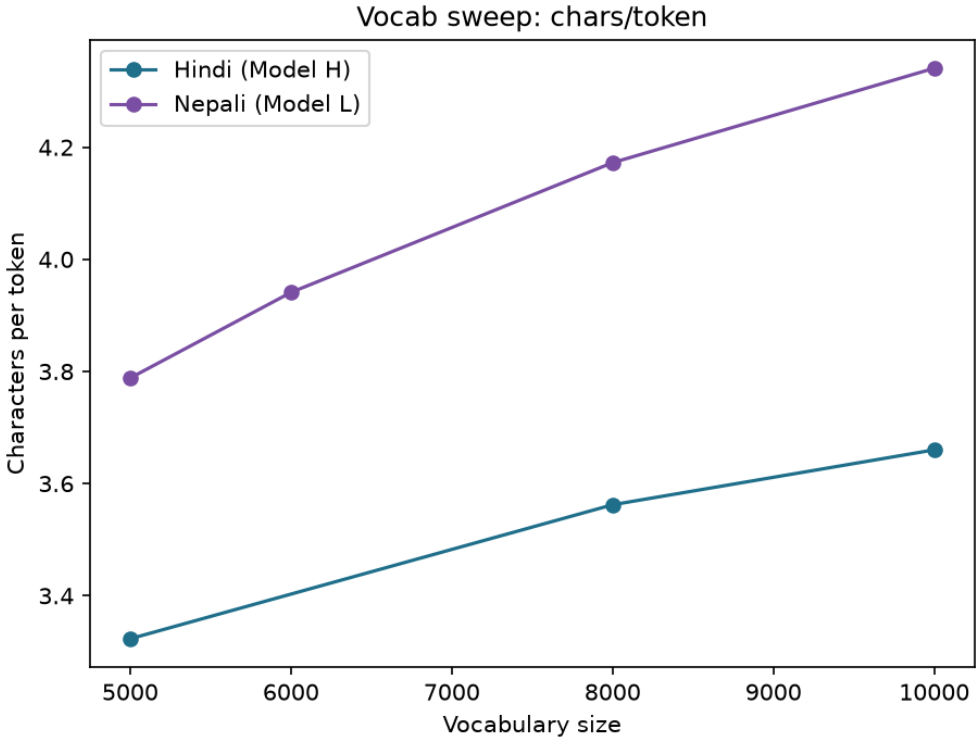


Figure 6.7: Characters per token across the swept vocabulary sizes, both languages.

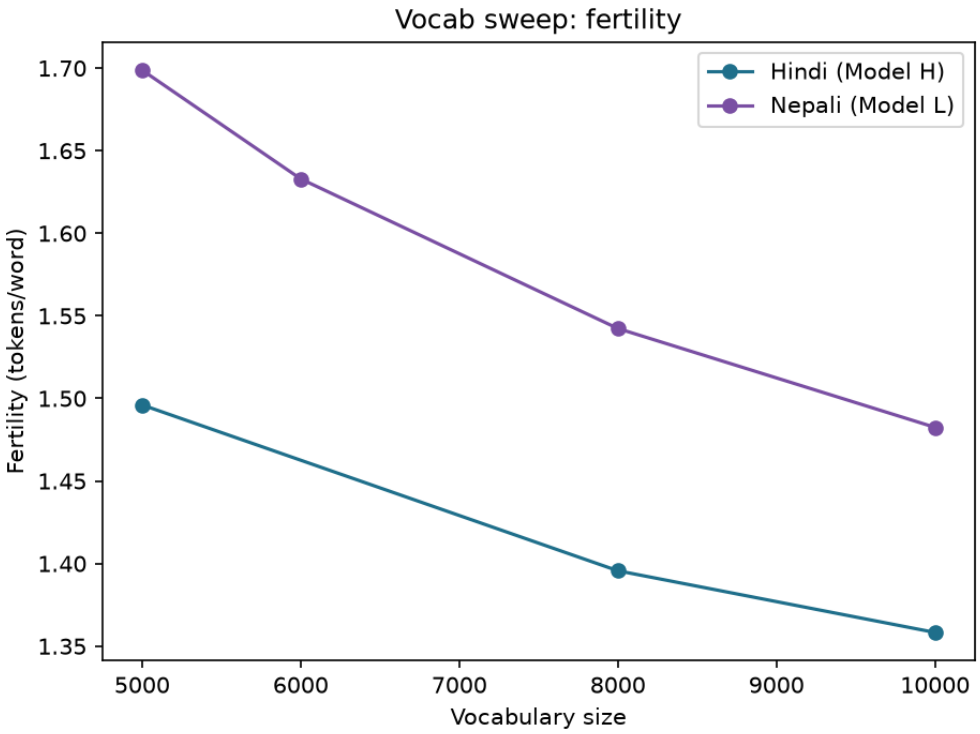


Figure 6.8: Fertility (tokens per word) across the swept vocabulary sizes – this is the curve each language's selection rule was actually reading.

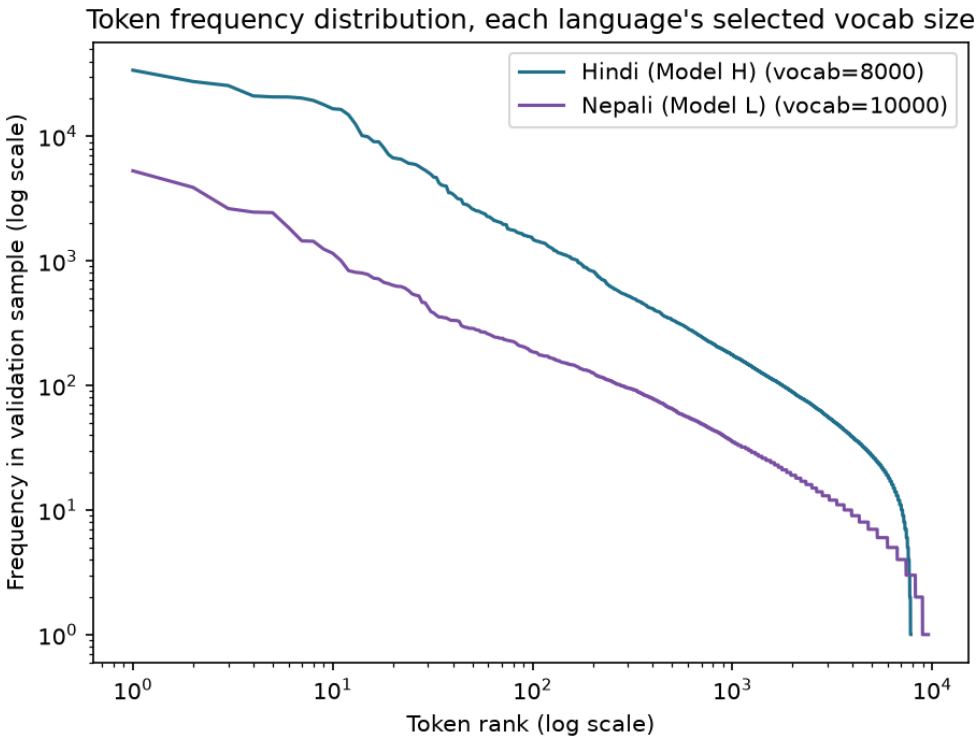
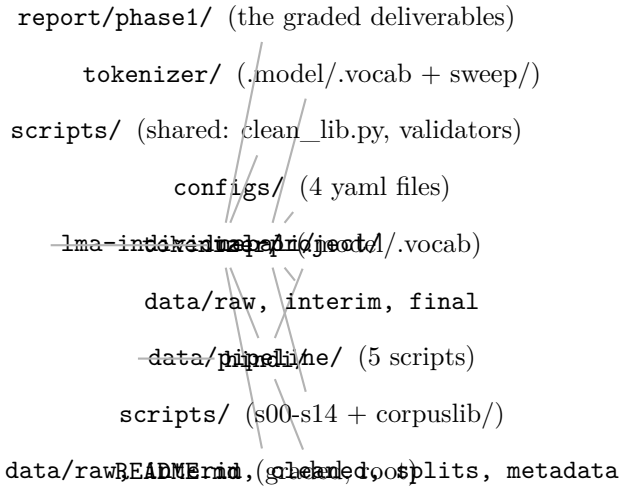


Figure 6.9: Token frequency distribution (log-log scale) at each language's own selected vocabulary size. The characteristic straight-line-ish shape is the Zipf pattern: a small number of very common pieces (punctuation, common postpositions) account for a large share of all tokens emitted, with a long tail of rare pieces.

Chapter 7

Repository Structure

7.1 Top-level layout



7.2 Where each stage's output actually lives

What	Hindi path	Nepali path
Raw manual	`hindi/data/raw/manual/`	`nepali/data/raw/manual/`
Raw downloaded	`hindi/data/raw/downloaded/`	`nepali/data/raw/downloaded/`
Post-normalize	`hindi/data/interim/`	`nepali/data/interim/s03_normali`
Per-stage snapshots	*(single pass, no per-stage dirs)*	`nepali/data/interim/s04_langid/`
Final cleaned	`hindi/data/cleaned/`	*(no single "cleaned" dir – final outpu*
Final splits	`hindi/data/splits/{train,validation,test}/`	`nepali/data/final/{train,val,te`
Tokenizer	`hindi/tokenizer/*.model`	`nepali/tokenizer/*.model` + `swee`
Stats/manifest	`hindi/data/metadata/clean_stats.json`	`nepali/data/final/corpus_stats.`

Notice the naming difference

Hindi's splits use the word `validation` (spelled out); Nepali's use `val` (abbreviated). This is not a typo anywhere – it's just that the two pipelines were never unified, so their naming conventions never got reconciled either. Both are correct within their own pipeline.

Chapter 8

Deliverables Mapping

This table is the direct answer to "which file satisfies which requirement" – exactly what a TA evaluation is likely to probe.

Requirement	**Hindi**	**Nepali**
Dataset collection scripts	`download_hindi.py`, `ingest_manual.py`, `ingest_wikipedia_dump.py`	`s01_collect.py` (driven by `collect.py`), `auto_grow_and_tokenize.py`
Preprocessing pipeline	`clean_hindi.py` + `scripts/clean_lib.py`	`s03`–`s10` (8 stage scripts) + `corpuslib`
Dataset statistics report	`report/phase1/phase1_report.md` (*same file, Nepali's own sections*)	
Train/val/test splits	`data/splits/{train,validation,test}.jsonl` + 3 manifest JSON files (too large for git – OneDrive)	`data/final/{train,val,test}.jsonl` + `split_stats.json` (too large for git – OneDrive)
Tokenizer training code	`train_tokenizer.py`	`s12_train_tokenizer.py`, `s13_tokenizer_eval.py`
Vocabulary files	`hindi_unigram_{5000,8000,10000,20000}.vocab` (committed to git)	`nepali_bpe.vocab` + 4 swept sizes (committed to git)
Tokenizer model files	`hindi_unigram_{5000,8000,10000,20000}.model` (committed to git)	`nepali_bpe.model` + 4 swept sizes (committed to git)

Honest gap, stated plainly

The actual split *data files* (Hindi's manifests, Nepali's `train.jsonl`/`val.jsonl`/`test.jsonl`) are **not committed to git** for either language – Hindi's train manifest alone is 469MB, and Nepali's three split files total roughly 9.2GB. Both are correctly excluded from git per the course spec ("do not commit large binary artifacts to git") and are meant to live on the shared OneDrive link in the root README instead. One small exception: Nepali's `split_stats.json` is only 763 bytes and *could* be committed for free, but currently isn't, purely because it's caught by the blanket `nepali/data/` gitignore rule.

Chapter 9

Running the Pipeline From Scratch

9.1 Hindi, step by step

```
python -m venv .venv
.venv/Scripts/activate
pip install -r requirements.txt

python hindi/data/pipeline/download_hindi.py
python hindi/data/pipeline/ingest_manual.py typed --input-dir <folder>
python hindi/data/pipeline/ingest_wikipedia_dump.py
python hindi/data/pipeline/clean_hindi.py
python hindi/data/pipeline/split_hindi.py
python hindi/tokenizer/train_tokenizer.py --vocab-size 8000
python hindi/tokenizer/analyze_tokenizer.py --vocab-size 8000
python scripts/validate_phase1.py
```

How to verify each stage worked: `clean_hindi.py` prints a summary (`docs_cleaned/docs_in`) and writes the full numbers to `data/metadata/clean_stats.json` – read that file, don't just trust the console. `validate_phase1.py` at the end is the single command that answers "is Phase 1 actually done" for Hindi – it checks all 4 required properties fresh from disk every time.

9.2 Nepali, step by step

```
cd nepali/scripts
python s00_doctor.py # ALWAYS run this first
python auto_grow_and_tokenize.py # runs S02-S14 in a loop until
                                 # both token targets are met
python ../../scripts/validate_phase1_nepali.py
python ../../scripts/check_nepali_hindi_overlap.py
```

Or, for finer control, run stages one at a time via `run_pipeline.py`:

```
python run_pipeline.py pipeline # S02 through S14, in order
python run_pipeline.py pipeline --from s08_dedup.py # resume from one stage
```

How to verify each stage worked: every Nepali stage writes a manifest JSON into its own `data/interim/<stage>/` folder with `docs_in`/`docs_out`/`retention` – read that, not just the

console output. `log.md` at the repo root accumulates one human-readable entry per stage run across the whole project's history.

9.3 Troubleshooting

Symptom	What it means / what to do
`BLOCKED: ... has no stage manifest`	The previous stage hasn't been run yet – Nepali stages check this explicitly before starting, rather than silently working on stale or missing input.
`another process already holds <lock>`	A previous run is still going, or died without cleaning up. If you're sure nothing is actually running, the lock self-expires after 15 minutes, or delete the `.lock` file manually.
`BLOCKED: retention X% outside expected [min, max]`	S04 or S05 aborted *on purpose* – a retention rate this far from expected is exactly the signature of the fastText/NumPy incident (Section 5.3): inspect the sample files under `data/interim/*_samples/` before trusting the run, don't just re-run and hope.
`no train text files found -- run split_hindi.py first`	Tokenizer training expects the splits to already exist; run the preceding pipeline stage first.
Resume takes a very long time	For Hindi, make sure `--resume` is actually being passed (a fresh run without it rebuilds from scratch). For Nepali, S08's near- duplicate pass is genuinely the slowest single stage in the whole pipeline (measured at 30+ hours on one production run) – this is not a bug, MinHash/LSH over millions of documents is just expensive.

Chapter 10

TA Evaluation Question Bank

Each question below is grounded in this project's actual work. "Simple answer" is what to say first, out loud; "Detailed answer" is the backup if asked to go deeper.

10.1 Theory

Q: What is a corpus, and why does this project need two of them?

Simple: A corpus is just the large collection of text a language model learns from. This project needs two, completely separate, because it's training two independent models – one per language – and the assignment forbids any sharing of data between them.
Detailed: Hindi's corpus (704,101 cleaned documents, 1.76B characters) and Nepali's (1,918,265 documents, 635M tokens) never share a document, never share a tokenizer, and are built by two completely different pipelines. This was explicitly verified: a cross-language dedup check found 0 documents shared between them.
Keywords: corpus, monolingual, independence, cross-language check

Q: Why does Unicode normalization matter for this project?

Simple: The same visual text can be stored as different byte sequences in Unicode; normalizing picks one canonical form so duplicate detection and cleaning actually work correctly.
Detailed: This project uses NFC, not NFKC, specifically because NFKC can strip or alter ZWJ/ZWNJ characters that control real spelling distinctions in Devanagari conjuncts – using NFKC would silently corrupt correctly-spelled Nepali/Hindi words.
Keywords: Unicode, NFC, NFKC, ZWJ, ZWNJ, Devanagari conjuncts

Q: Explain MinHash and LSH in your own words.

Simple: MinHash turns a document into a short numeric "fingerprint" such that similar documents get similar fingerprints. LSH groups documents by fingerprint so only likely-similar documents ever get compared directly, instead of comparing every document to every other one.
Detailed: A document is broken into overlapping word-shingles; 100 different hash functions are applied and only the minimum result of each is kept, forming a 100-number signature. Two

documents' true word-overlap similarity is approximated by what fraction of their signature numbers match. LSH splits the signature into 20 bands of 5 numbers; documents are only compared if they share at least one full band – this makes near-duplicate detection scale to millions of documents instead of requiring an all-pairs comparison.
Keywords: MinHash, LSH, shingle, Jaccard similarity, near-duplicate

Q: What's the difference between BPE and Unigram tokenization?

Simple: BPE builds a vocabulary by repeatedly merging the most frequent adjacent pair of tokens, and never undoes a merge. Unigram starts from many candidate pieces and uses probability to prune down to the target size, reconsidering the best segmentation for each piece of text.
Detailed: BPE's greedy merging is fast but permanent – an early merge inside a bad boundary can't be revisited. Unigram treats segmentation itself as probabilistic, which is argued (though not empirically confirmed on this data) to suit Devanagari's morphologically rich, conjunct-heavy structure better. This project uses both: Hindi uses Unigram (a considered choice), Nepali uses BPE (this pipeline's configured default), with an automatic fallback to also try Unigram if BPE's fertility got too high (it never triggered for Nepali).
Keywords: BPE, Unigram, SentencePiece, subword tokenization, fertility

Q: Why is data leakage between train and test a serious problem?

Simple: If test data (or something too similar to it) leaks into training, evaluation results look better than the model's real capability – you're no longer measuring generalization to unseen data.
Detailed: This project guards against leakage at two levels: document-level (never split by sentence, since that leaks context across splits) and near-duplicate-level (Nepali's S09 decontamination stage specifically removes train documents that are near-duplicates of held-out val/test content, verified by a rescan finding zero remaining matches).
Keywords: leakage, generalization, decontamination, near-duplicate

10.2 Pipeline

Q: Why does Nepali's S07 (holdout) run before S05/S06 despite the file numbers?

Simple: If the same quality filters that pick training data also get to curate the evaluation set, evaluation partly measures the filters' own preferences instead of genuine model quality.
Detailed: S07 forks val/test out of the pool right after S04 (normalize + language-ID only). S05 (heuristics) and S06 (perplexity) both read S07's leftover `train_pool` output, never S04's output directly – so val/test are never touched by any quality-filtering decision, keeping them a fair, unbiased sample of what was actually collected.
Keywords: holdout, execution order, unbiased evaluation, train_pool

Q: Walk through what happens to one document from collection to being tokenizer training data, for Nepali.

Simple: Collected → normalized → language-checked → (if selected for train) quality-filtered → perplexity-filtered → deduplicated → decontamination-checked → lane-balanced → written to `train.jsonl` → used to train the tokenizer.
Detailed: S01 collect → S03 normalize (NFC, doc_id becomes a content hash of normalized text) → S04 langid (fastText margin check) → S07 assigns it to train/val/test by a seeded hash of its doc_id → if train: S05 heuristics → S06 perplexity → S08 exact+near dedup → S09 decontamination check against held-out val/test → S10 lane balancing (only affects downloaded-lane documents) → S11 writes it into `train.jsonl` → S12 trains the tokenizer using it.
Keywords: pipeline stages, doc_id, content hash, stage order

Q: Why does Hindi's pipeline process manual documents before downloaded ones?

Simple: So that when two copies of the same text exist (one manual, one downloaded), deduplication always keeps the manual copy.
Detailed: `clean_hindi.py`'s `RAW_DIRS` list is explicitly `[raw/manual, raw/downloaded]`. Deduplication keeps the first-seen copy of a duplicate hash and drops later ones – so this ordering is what guarantees a manual/downloaded collision always resolves in favor of manual, protecting the scarce resource that the 20% floor is measuring. Nepali achieves the same guarantee differently: its dedup stage explicitly checks lane and prioritizes manual regardless of processing order.
Keywords: dedup priority, manual lane, RAW_DIRS ordering

Q: What does `s00_doctor.py` actually protect against?

Simple: A silent environment bug that once deleted 94% of a corpus while logging "SUCCESS."
Detailed: `fasttext` crashed on a NumPy version incompatibility; the crash was caught by a blanket `except Exception` and treated as "document rejected," so nearly the whole corpus was silently thrown away over four hours, with the run reporting success throughout. `s00_doctor.py` now tests every dependency actually works *before* any expensive stage runs, and the retention-band abort in S04/S05 is the second layer of defense against the same failure mode recurring in a different form.
Keywords: silent failure, fastText, NumPy, retention band, fail loudly

10.3 Code and files

Q: What is `corpuslib`, and why does it exist as a separate package?

Simple: It's the shared code every Nepali pipeline stage imports – config loading, JSONL I/O, dedup math, language ID, etc. – kept in one place so 15 stage scripts don't each reimplement the same logic.
Detailed: Modules include `config.py` (loads YAML configs into one object), `stageio.py`

(reads/writes a stage's JSONL + manifest), `dedup.py` (MinHash/LSH, used by both S08 and S09), `langid.py`, `heuristics.py`, `ngramlm.py`. It was originally designed to be shared between both languages (with Hindi as the intended "source of truth" copy, per the `sync` subcommand in `run_pipeline.py`), but only Nepali actually uses it in this repo.
Keywords: corpuslib, shared library, stageio, config loading

Q: What does `scripts/validate_phase1.py` actually check?

Simple: Four things for Hindi: raw manual+downloaded sources exist, manual share is at least 20%, splits exist with no overlap, and a tokenizer has been trained.
Detailed: Each check reads real files (never a hardcoded count): `raw/{manual,downloaded}/metadata.jsonl` for source presence, `cleaned/metadata.jsonl`'s `source_type` field for the manual percentage, the three split manifests for a document-id intersection check, and a glob for any `.model` file. It originally also checked a Hindi↔Bhojpuri cross-language overlap; that check was removed once Bhojpuri's files were deleted from the repo (Nepali has its own separate cross-language script against Hindi instead).
Keywords: validator, PASS/FAIL, manual floor, no-leakage check

Q: Where is the manual-vs-downloaded distinction actually recorded in the data?

Simple: A `source_type` field ("manual" or "downloaded") on every single document's metadata, from the moment it's collected.
Detailed: This field is written at collection time (e.g. `download_hindi.py` always writes `source_type= "downloaded"`; `ingest_manual.py` always writes `"manual"`) and survives unchanged all the way through cleaning into the final corpus, which is what lets the manual percentage be recomputed directly from real files at any time rather than trusted as a remembered number.
Keywords: source_type, metadata, provenance, manual percentage

10.4 Data and design decisions

Q: Why absolute token targets instead of a manual/downloaded ratio for Nepali?

Simple: A ratio target conflates "how much text exists" with "what fraction is manual" – an earlier version of this exact idea threw away a huge amount of good downloaded text just to hit a percentage, when the manual lane was genuinely small.
Detailed: Chasing a ratio (e.g. "manual must be 24% of total") when manual was only 5.82% forced cutting downloaded from 209.8M words to 41.1M – destroying volume to satisfy a percentage. This project's actual approach: two independent absolute targets (manual $\geq$ 100M tokens, downloaded $\geq$ 400M tokens). Manual is kept at 100% always; downloaded is trimmed to its own target; if manual still falls short, that's a signal to collect more manual data, never a reason to cut downloaded further.
Keywords: absolute target, ratio, lane balancing, S10

Q: Why is Nepali's manual share reported as token-weighted while Hindi's is char-weighted?

Simple: Because Nepali's pipeline never added per-lane character tracking – token-weighted is the real, honest number that's actually available for it.
Detailed: This is disclosed explicitly in the report rather than hidden or forced into a false apples-to-apples comparison. It's one of several genuine methodology differences between the two pipelines (see also: different dedup methods, different tokenizer algorithms).
Keywords: token-weighted, char-weighted, methodology difference

Q: Why does this project not use a pretrained tokenizer or model?

Simple: The assignment explicitly forbids it – the entire point of the exercise is to build everything from scratch to demonstrate real understanding.
Detailed: No pretrained language models, no pretrained tokenizers, no HuggingFace model classes are allowed anywhere in this project, for either phase. This is why even the tokenizer training (SentencePiece) starts from raw cleaned text with no prior vocabulary, rather than extending or fine-tuning an existing one.
Keywords: from-scratch, no pretrained models, assignment constraint

10.5 Hindi-specific

Q: Why does Hindi's cleaning pipeline have a Bhojpuri-marker check at all?

Simple: Because `langdetect` (the library Hindi's pipeline uses) has no label for Bhojpuri, so it would happily call real Bhojpuri text "Hindi" – the marker check catches what the library structurally cannot.
Detailed: `clean_lib.py`'s `marker_counts()` counts occurrences of common Bhojpuri words (बा, बाटे, गइल, etc.) versus common Hindi words (है, हैं, गया, etc.); if Bhojpuri markers clearly outnumber Hindi markers, the document is rejected as `likely_bhojpuri` *before* `langdetect` even runs. This filter predates and is independent of Bhojpuri's later removal as Model L – it's part of Hindi's own cleaning history and still runs today, even though the risk it originally guarded against (accidentally training Hindi on the language that used to be this project's own Model L) no longer applies day to day.
Keywords: likely_bhojpuri, marker_counts, langdetect limitation

Q: What was the Hindi↔Bhojpuri cross-language incident, exactly?

Simple: 2 documents were found to be genuinely Bhojpuri text that an upstream dataset (Sangraha) had mislabeled as Hindi; both were removed from Hindi's cleaned corpus with a small, targeted, one-time script.
Detailed: `validate_phase1.py`'s cross-language check (while Bhojpuri was still Model L) found 2 documents sharing a content hash between the Hindi and Bhojpuri cleaned corpora. Both contained Bhojpuri-specific verb forms not in Hindi's marker list, and had been sourced as `hin` by `ai4bharat/sangraha` while correctly labeled `bho_Deva` by `HuggingFaceFW/fineweb-2` for the same underlying text – an upstream labeling error, not a bug in this project's pipeline.

`scripts/remove_cross_language_duplicates.py` removed them, which is why Hindi's final count is 704,101, not `clean_stats.json`'s own recorded 704,103.
Keywords: cross-language contamination, upstream labeling error, 704101 vs 704103

10.6 Nepali-specific

Q: Why does Nepali use BPE if the theoretical argument (in this project's own Hindi section) favors Unigram for Devanagari?

Simple: Because BPE was this pipeline's configured default, chosen independently before that Hindi-specific argument was written up – not a per-language linguistic decision. There's a real safety net though: a comparison to Unigram would have kicked in automatically if BPE's fertility got too high, and it simply never needed to.
Detailed: `nepali/configs/tokenizer.yaml` sets `model_type: bpe` with `compare_unigram.trigger_fertility_above: 2.10`. Nepali's measured BPE fertility across the whole sweep (1.48–1.70) never came close to that threshold, so the automatic Unigram comparison never triggered. This is a real, working comparison mechanism that happened not to change the outcome – not an untested assumption.
Keywords: BPE vs Unigram, compare_unigram, fertility threshold

Q: Why did Nepali's pipeline run on a separate machine?

Simple: The MinHash/LSH near-duplicate dedup stage (S08) alone took over 30 hours on this project's primary machine, which wasn't practical within the deadline window.
Detailed: Only the pipeline's output artifacts were transferred back into this repository, not live per-stage console logs – so `log.md`'s later-stage entries for Nepali (S08 production run onward) are reconstructed after the fact from those output files, which is disclosed explicitly in the log itself rather than presented as if it were live-logged.
Keywords: provenance, separate machine, reconstructed log entries

Q: Why was Bhojpuri replaced by Nepali as Model L?

Simple: Bhojpuri was fully built and passed every check, but its real available public data is genuinely small (its final corpus reached only 16.8% of the 500M-token target, a documented and spec-permitted shortfall). Nepali had enough available public data to clear the same target comfortably, and once its own pipeline was finished, it replaced Bhojpuri.
Detailed: Bhojpuri's shortfall was not a collection-effort problem – every known manual source was recrawled and confirmed exhausted, and every findable Bhojpuri-labeled Hugging Face dataset was evaluated. Nepali's larger public-data availability (via AI4Bharat's Sangraha and IndicCorpV2) let it reach 127.0% of target instead. This was a deliberate, disclosed switch, not a quiet substitution – Bhojpuri's own numbers remain in the report as history, and its code has since been removed from the repository, but nothing about that history is hidden.
Keywords: Model L switch, Bhojpuri shortfall, spec-permitted

10.7 Limitations

Q: What is the single biggest acknowledged weakness in this project's cleaning?

Simple: Hindi's language-ID/contamination check is a hand-written lexical marker list, not a trained classifier – it will not catch every case, and the cross-language incident (2 leaked documents) is proof it already missed some.
Detailed: The report states this honestly rather than overselling the filter's reliability: a coarse lexical heuristic can only ever be a partial defense. Given more time, a real trained classifier would be the first thing to build instead.
Keywords: lexical heuristic, known limitation, trained classifier

Q: What's the biggest structural difference between the two pipelines that was never reconciled?

Simple: Near-duplicate detection – Hindi only does exact-hash dedup; Nepali runs full MinHash/LSH near-duplicate detection. This is a real asymmetry, not a shared gap.
Detailed: Hindi's pipeline was built first and never had MinHash/LSH added to it, even after Nepali's pipeline demonstrated it works. Given more time, this would be a natural next improvement for Hindi's pipeline specifically.
Keywords: exact-hash, MinHash, pipeline asymmetry

Chapter 11

Cheat Sheet

11.1 Numbers to have memorized

	Hindi (Model H)	**Nepali (Model L)**
Final tokens	505,852,028 (101.2%)	635,204,333 (127.0%)
Final documents	704,101	1,918,265
Manual share	28.65% (char)	26.41% (token)
Train/val/test	675,937/14,082/14,082	1,520,000/199,428/198,837
Tokenizer	Unigram, vocab 8,000	BPE, vocab 10,000
Dedup method	exact-hash only	exact-hash + MinHash/LSH
Cross-language overlap	0 shared documents (verified)	

11.2 File names to have memorized

Hindi	Nepali
`download_hindi.py`	`s01_collect.py` / `collect.py`
`clean_hindi.py` + `clean_lib.py`	`s03`–`s10` + `corpuslib/`
`split_hindi.py`	`s07_holdout.py` + `s11_finalize_splits.py`
`train_tokenizer.py`	`s12_train_tokenizer.py`
`analyze_tokenizer.py`	`s13_tokenizer_eval.py`
–	`auto_grow_and_tokenize.py` (outer loop)
`scripts/validate_phase1.py`	`scripts/validate_phase1_nepali.py`

11.3 Pipeline stage order to recite

Nepali: **S00 doctor → S01 collect → S02 calibrate → S03 normalize → S04 langid → S07 holdout → S05 heuristics → S06 perplexity → S08 dedup → S09 decontaminate → S10 balance lanes → S11 finalize splits → S12 train tokenizer → S13 tokenizer eval → S14 corpus stats.** Remember: S07 runs out of order, right after S04, before S05/S06 – this is the single most-likely-to- be-asked-about sequencing detail in this whole project.

Hindi: **download → ingest (manual + Wikipedia dump) → clean → split → train tokenizer → analyze tokenizer.** No stage numbering, no orchestrator script – run by hand, in

this order.

11.4 Design decisions to be ready to defend

- Manual copy always wins a manual/downloaded duplicate tie – protects the scarce resource the 20% floor measures.
- Absolute token targets (Nepali), not a manual/downloaded *ratio* – avoids conflating volume with composition.
- Val/test held out *before* quality filtering (Nepali S07) – keeps evaluation unbiased by the filters' own preferences.
- NFC, never NFKC – preserves real Devanagari spelling (ZWJ/ZWNJ matter).
- Document-level splits, never sentence-level – prevents leakage.
- Fail loudly, not silently – S00/S04/S05's abort-on-bad-retention behavior exists because of a real incident that deleted 94% of a corpus under a "SUCCESS" log.

11.5 One-sentence summary, if asked "what did you actually build in Phase 1?"

Recite this

"I built two independent data pipelines – Hindi's is a single-pass cleaner, Nepali's is a 15-stage pipeline with its own dedup and quality filters – that together turned 3.4 million raw documents into two clean, split, tokenized corpora totaling over 1.1 billion tokens, both clearing the 20% manual-collection floor, with zero content overlap between the two languages, verified end to end."

Chapter 12

Honest Limitations Summary

This chapter exists so nothing is quietly overstated. All of these are also disclosed in the actual graded report.

- **Hindi has no near-duplicate detection**, only exact-hash – Nepali's MinHash/LSH capability was never retrofitted onto Hindi's pipeline.
- **Hindi's Unigram-vs-BPE argument is structural, not empirically tested** on Hindi's own data specifically.
- **The Bhojpuri-contamination marker list is a coarse heuristic**, not a trained classifier, and is known to have missed real cases (the 2-document cross-language incident).
- **Nepali's manual share is token-weighted**, not char-weighted like Hindi's – a genuine methodology difference, not an oversight glossed over.
- **Nepali's SentencePiece sampled-character count was not logged** by its pipeline the way Hindi's was – a real reporting gap.
- **The cross-language overlap check only covers Nepali vs. Hindi**, not Nepali vs. Bhojpuri, since Bhojpuri is no longer part of the submission.
- **Whether the large data files are actually uploaded to OneDrive** (as opposed to just existing locally) was not independently verifiable at the time of writing this project's documentation.
- **No neural network architecture exists yet.** Phase 1 is data and tokenizer only – this is by design, not an oversight, but worth stating plainly if asked.